Making LLM Inference Extensions Compose: Contracts for Interfaces, Dependencies, and Runtime Effects

Anonymous Submission

Abstract

LLM inference extensions enter through incompatible integration surfaces: package entry points, private host hooks, direct patches, and extracted policy modules. Their dependencies extend beyond package versions to host symbols, configuration predicates, import order, process scope, and shared runtime state. Consequently, individually installable extensions need not form a working composition. We study how to make these requirements explicit and checkable without treating successful loading as proof of compatibility. ECPA combines extension contracts, a conflict-aware plan compiler, typed host adapters, and authority-owned runtime evidence. Contracts describe interfaces, dependencies, resource ownership, and effect obligations; adapters bind those declarations to concrete execution paths. The compiler checks declared constraints, while runtime evidence is required for claims that static checks cannot establish. A source audit distinguishes original integration behavior from ECPA-added descriptors and identifies both representable requirements and unresolved adaptation work. We redesign the composition evaluation around independently labeled source cases, installation-order controls, real effect checks, and measured adaptation effort. This expanded study is planned, not a completed result. Existing evidence comprises a 4-case definite H2 subset and an earlier nine-start H1 study that falsified the tested implementation's prevention claim. The current prototype therefore does not yet establish general composability, recovery safety, or bounded overhead.

1 Introduction

Inference systems increasingly depend on independently developed extensions for scheduling, KV-cache management, speculative decoding, communication, and observability. Operators want to combine these extensions on one serving stack. Yet the integration unit is often not a stable interface. One MOD registers a plugin, another wraps a private scheduler method, and a third provides a policy that still needs a host executor. Package dependency resolution cannot determine whether these implementations can work together.

Our central question is: *how can a serving system turn heterogeneous extension interfaces and implicit dependencies into a composition whose requirements, conflicts, and runtime effects are explicit and checkable?* The challenge begins before failure recovery: an extension must first have a concrete integration path, and the composition must preserve that path's preconditions and ownership boundaries.

The real sources expose three connected problems. First, an installable package may require undocumented or separately documented host symbols and configuration. KNorm declares no package dependencies, but its activation guard enumerates private host surfaces; its compression predicate also requires an explicit environment switch, a compression ratio below one, and disabled prefix caching. Second, different resource names can hide the same execution path. KNorm and KVCompress both wrap `Scheduler.update_from_output`; a shared symbol is a reason to inspect ordering and state interaction, not by itself proof of a conflict. Third, a discoverable descriptor may not have an executable host integration. The audited QoS, split-batch, and unified-communication snapshots expose import-only components and proposed host contracts. Discovering those descriptors does not activate the corresponding scheduler, graph executor, or native collective.

We build ECPA to make these distinctions part of extension composition. A contract describes capabilities, host requirements, lifecycle authority, resource claims, process scope, and observable effects. A compiler produces an immutable plan and checks declared dependencies and ownership. A typed adapter binds that plan to the selected host, and runtime authorities provide evidence for the obligations they can actually observe. The decision is therefore about a concrete host, provider set, configuration, and topology, not just whether a package is installed.

This design has an important boundary. ECPA does not automatically infer arbitrary MOD semantics, supply a miss-

ing host hook, or prove that two wrappers commute. Contracts are audited inputs rather than ground truth. Known incompatibility may justify rejection; missing knowledge must remain unverified. An ECPA adapter that adds a new integration seam is measured as adaptation work, not counted as a capability of the original MOD.

Figure 1 shows the resulting separation between contracts, planning, host execution, and evidence. Our contribution is organized around the full composition problem:

- a source-grounded account of heterogeneous integration surfaces, implicit prerequisites, and shared-state hazards in real inference MODs;
- a contract, compiler, and runtime that express and check known requirements while separating declaration, invocation, and effect; and
- a falsifiable composition study that separates independent labels from predictions, tests both orders and singleton controls, and accounts for unsupported cases and adaptation cost.

The first contribution is a source audit; the expanded study is not yet complete. We evaluate one vLLM-compatible host family and do not claim cross-framework validation. Existing H2 evidence has only 4 definite cases. The earlier matched H1 experiment found that all three ECPA repetitions retained an effective claim at 0.5 process coverage, falsifying that implementation's prevention claim. Recovery (H3) and overhead (H4) remain supporting validation, with no completed formal results. Release 0.3.0 and Manifest 0.3 describe implementation status, not proof that the proposed composition guarantees have been achieved.

2 Background and Motivation

2.1 Why Existing State Is Ambiguous

A package manager proves that files exist. Entry-point discovery proves that metadata contains a name. Configuration proves that an operator supplied parameters. An enabled bit proves desired intent. None proves that the selected implementation executed in every process that matters. Conversely, an external service can become unreachable while enabled intent should remain for diagnosis and later recovery. A linear state machine both overclaims success and destroys useful failure context.

Vanilla entry points provide a deliberately lightweight extension mechanism [24]. Manual integration can add flags, environment variables, and operator checks. Both are legitimate baselines. Their limitation is not that they can never be made safe; it is that evidence obligations, ownership, and rollback are convention rather than a common machine-checkable contract.

2.2 From Source Requirements to Composition Obligations

The new audit preserves exact source revisions and distinguishes three levels of evidence: source-observed requirements, independently observed execution outcomes, and unresolved composition hypotheses. The frozen 24-MOD page population remains the sampling frame; newer revisions are separate observations, not replacements for its historical counts. Host and backend dependencies are supporting sources, not extra MODs.

For example, KNorm's source checks an in-tree duplicate and required host modules, gates compression on configuration, and rebinds already-imported aliases. ECPA can represent the known prerequisites and bind invocation obligations, but it cannot claim to discover every alias or prove a useful compression effect merely from registration. Similarly, an import-only communication descriptor needs a real provider before its process-group ownership contract can be exercised. These gaps guide the experiment rather than being filled with invented compatibility labels. The source-to-contract audit records each issue, its existing ECPA mechanism, required adaptation, and remaining verification obligation. A concrete source-level exclusion illustrates the need for topology predicates: the audited Pipeline policy requires PP greater than one, while the audited KVCompress stateful adapter requires PP equal to one. Their supported configuration domains do not intersect; this source fact is separate from a native conflict result. A separate CPU ABI preflight also found that an Ascend runner transitively imports a Host model-config module absent in a newer Host revision, despite accepted version metadata. That preflight is a tool observation, not evidence that the ECPA compiler automatically discovered the hidden dependency.

3 Problem and Formal Model

3.1 System Model

We model a host runtime H, providers P, plugins G, processes W, and resources R. A plugin manifest declares required and provided capabilities, version predicates, claimed resources, provider needs, and evidence obligations. A provider translates an accepted plan into host-specific configuration, but it does not supervise the host or acquire authority over an external service. Each launch has a unique identity ℓ and an expected process set W_ℓ.

An activation plan $A = (H, P, G, W, R, V)$ is admissible only if capability requirements resolve under a frozen public-PEP-440 comparison profile, resource ownership is conflict-free or mediated by a selected claimant, and every evidence obligation names an exact role/event/direction/capability authority grant and its selected capability provider plus, when applicable, a positive quorum threshold. Let $O_A^{required}$ be the

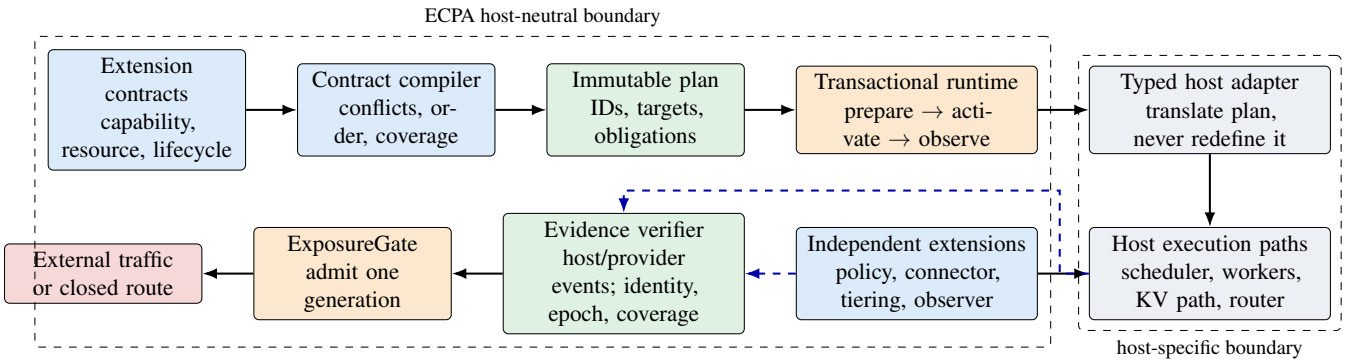


Figure 1: ECPA separates portable extension semantics from a narrow host adapter. The adapter maps an immutable plan into native execution paths; it cannot declare success. Effect evidence returns from the authority that can observe each obligation, and only the ExposureGate can make a generation visible.

required process, resource, provider-operation, and external-authority obligations compiled into A, and let E be the observed evidence set. Runtime effectiveness is:

$$effective(A,\ell,E) \iff \forall o \in O_A^{required} : complete(o,A,\ell,E).$$

Each completion predicate checks the obligation's authority, identity, freshness, and value; process obligations additionally check membership or an explicit positive quorum. The manager may not silently remove an obligation or weaken its predicate.

ECPA deliberately separates five facts that conventional extension state often collapses. $D(p)$ means a contract declares extension p; $P(p,A)$ means an admissible immutable plan selects it; $I(p,w,A,\ell)$ means a plan-bound host path invoked it in process w; $F(A,\ell,E)$ abbreviates the effectiveness predicate above; and $X(A,\ell)$ means the ExposureGate admitted the generation to external traffic. Compilation ensures $P \Rightarrow D$, valid invocation evidence ensures $I \Rightarrow P$, and the gate enforces $X \Rightarrow F$. No reverse implication is assumed, and F does not reduce to I: a plan can also require resource effects, provider operations, and external-authority facts. Loading is therefore an activation event, not an effectiveness verdict. A false-effective outcome is

$$reported_effective(A,\ell) \wedge \neg F(A,\ell,E),$$

where the right-hand side is computed from the preregistered target and obligation sets together with authority-owned observations, rather than from the manager's status bit.

3.2 Threat Model

We address accidental incompatibility, stale or replayed evidence, conflicting ownership, provider ambiguity, parent/worker version mismatch, partial rollout, observer loss, process crashes, and interrupted rollback. Plugins and providers may be buggy and may report inconsistent metadata. We assume the host, package database, and process identity channel are administratively trusted. We do *not* sandbox malicious Python, defend a compromised host, or claim that signed metadata makes arbitrary plugin code safe. This boundary follows the upstream warning that plugins execute with application authority [25]. The formal harness trusts the experiment host, runner, registered lifecycle sources, and registry-pinned observer, but not SUT acknowledgements or telemetry. It does not defend against malicious same-user code inspecting descriptors or rewriting artifacts. Five distinct sources own readiness, workload, fault, host-effect, and shutdown facts. Their challenge-bound receipts carry plan/launch/invocation and PID/start-time identities over kernel-credentialed Unix channels; an ACK alone proves none of these facts. Formal observables carrying `causal_ack` are rejected. The host observer reads a host-owned journal and cannot stand in for the other sources. Fixture services and their artifacts cannot enter the formal aggregate. These checks make records runner-consistent within the stated trust boundary, not cryptographic attestations against a malicious SUT.

Table 1: Extension facts and the authority permitted to establish them.

Fact	Evidence	Owning authority
Declared	static manifest and distribution metadata	package registry
Planned	immutable plan and resolved provider graph	contract compiler
Invoked	launch-bound hook or dispatch event	host execution path
Effective	every required obligation completes	obligation authorities
Exposed	generation admitted to external traffic	ExposureGate

3.3 Invariants and Failure Semantics

ECPA enforces four invariants. **I1, evidence monotonicity:** a stronger state requires new evidence and cannot be inferred from a weaker state. **I2, single explicit ownership:** every exclusive resource has one owner or a declared mediator. **I3, authority locality:** evidence is accepted only from the component that can observe the claimed event. **I4, rollback preservation:** no new plan destroys the last committed valid plan before the replacement satisfies its commit gate.

Failures are structured outcomes, not booleans. A missing provider or unsupported version makes a plan inadmissible. A service outage can preserve *enabled* intent while marking health degraded and refusing a new launch. A missing worker attestation leaves the activation partial, never globally effective. Stale evidence is rejected by launch, process, host, manifest, and runtime-version identity. Rollback either restores the recorded predecessor or reports a bounded operator action; it never invents success.

3.4 Safety Argument

Assume that plan publication and generation compare-and-swap are atomic, each evidence source belongs to its declared administrative authority, the channel preserves sender identity, and target membership is frozen for one process epoch. Then an ECPA-controlled route cannot expose a candidate generation before all of its required obligations are satisfied. The planner fixes the obligations and target set in the plan digest. Evidence validation rejects a different plan, launch, controller, process epoch, target identity, or invocation sequence. The ExposureGate opens only for that digest after the validator reports a complete set, and its route fence names the same generation. A delayed validator or controller therefore fails the generation compare-and-swap rather than opening a newer route.

This conditional argument concerns admission, not functional correctness. A source closure inside an acknowledged prefix bounds a finite observation; it does not keep an effect valid after process replacement or evidence expiry. Continuing admission requires epoch fencing and an explicit invalidation policy. False semantic events from authorized sources or undeclared effects invalidate the premises. Rollback closes the candidate before restoring its predecessor; it cannot restore external state the manager does not own. These boundaries require independent observers and fault injection.

We also encode the declared exposure transaction as a finite abstract model. Exhaustive fixed-point exploration reaches 39 states and 102 valid edges, while rejecting 522 invalid state-action pairs. None of the reachable states violates its 7 checked invariants, including no candidate admission before a proof-bound open receipt, no admission in failed-safe or safety-unknown states, and predecessor-only strong rollback. This is evidence about the stated abstraction under its recorded assumptions, not a proof that the implementation refines the model or that a real prototype-host deployment recovers correctly. H3 therefore remains a planned formal-real claim.

3.5 Refinement Boundary

The safety claim spans three representations that must not be conflated. The contract compiler produces a canonical plan and obligation set; the durable coordinator records an intended state transition; and the traffic authority controls the externally observed route. Let durable coordinator state be $S = (g, q, p, c, f, \pi)$, where g is the candidate generation, q the lifecycle state, p and c the predecessor and candidate digests, f the route fence, and π the accepted proof digest. Let the traffic authority report $R = (g_r, f_r, d_r)$ for its observed generation, fence, and route snapshot digest. The abstract predicate *exposed* is represented only when

$$q = \textsc{CandidateOpen}, \qquad (g_r, f_r, d_r) = (g, f, c), \qquad \mathit{valid}(\pi, g, c).$$

A database row alone is therefore not an exposure fact, and a route mutation alone is not a committed transition.

The reference implementation uses an intent–effect–query–receipt protocol at every authority-crossing transition. It first commits a transition intent, invokes the traffic adapter, queries the adapter's actual route, and commits a receipt only if generation, fence, and snapshot digest match. Recovery treats an intent without a receipt as an uncertain operation and reconciles it from a fresh route query. A failed or unobservable close enters SAFETYUNKNOWN rather than manufacturing fail-closed success. Table 2 states the refinement obligations explicitly; passing the finite abstract model establishes none of them by itself.

These obligations separate the theorem we want from the implementation we have. R1–R5 are exercised today only by the single-host reference gate and its fault-injection suite. Establishing them for the prototype host requires the registered formal-real adapter, host-owned process evidence, and a real traffic authority. Until then, the artifact supports an abstract safety result and a reference-implementation conformance result, not an end-to-end refinement claim.

4 Design

4.1 Typed Static Manifests

Discovery reads distribution metadata and a static manifest without importing implementation modules. The schema distinguishes bundle identity, component kind, semantic capability, host/API range, provider, required service, resource claims, and activation/evidence contracts. Static discovery

Table 2: Refinement obligations between the abstract transaction and the reference implementation.

Obligation	Implementation boundary and falsification test
R1: plan immutability	Bind the canonical plan and target digest before activation. Falsify with mutation and mixed-generation inputs.
R2: evidence binding	Insert a receipt only after identity, authority, epoch, and coverage checks. Falsify with replay, source substitution, and PID reuse.
R3: exposure agreement	Query the traffic authority and match the proof-bound generation, fence, and snapshot before an open receipt. Crash at each intent/effect/query boundary.
R4: stale-writer exclusion	Bind a lease token and durable revision compare-and-swap. Recreate the manager, expire the lease, and delay a writer.
R5: recovery conservatism	Reconcile unmatched intents from the traffic authority and block unknown routes. Lose the query and fail compensation.

avoids executing plugin code merely to decide whether it is eligible. Namespace ownership is typed and parameterized by the host profile: a manifest names the owning authority and exact registration group; an adapter may delegate to a host-native hook but may not mint a group owned by another authority. No namespace or package prefix is built into the ECPA schema.

The namespace incident becomes a schema fixture: package metadata must contain exactly the declared group and bundle identity, and a clean installed-package probe must observe it. This closes the test gap that allowed a constant to change while mock-based tests continued to pass.

4.2 Capability Negotiation

Capabilities name semantics rather than package names. A plugin can require a versioned preemption-policy interface, a connector contract, or process-owned invocation evidence. The host and providers advertise concrete capabilities with provenance. Negotiation produces a graph whose edges explain which authority satisfies each requirement. Missing, ambiguous, or incompatible edges reject activation. Untested version tuples remain *unverified*, not implicitly compatible.

Contract acquisition and its limits. The current prototype consumes manually supplied contracts and adapters. It does not infer hidden dependencies from arbitrary source or learn conflicts from package names. The audit therefore traces each declaration back to original source and records separately any ECPA-added requirement, resource alias, adapter, or host modification. Static acceptance is conditional on those inputs; unknown interfaces and missing effect evidence do not become compatibility facts. Automatic contract extraction is not an implemented contribution of this paper.

4.3 Conflict Planning

Plugins claim resources such as a command-line/configuration key, hook slot, scheduler policy, connector, service endpoint, or deployment field. The planner constructs an ownership graph and detects duplicate exclusive claims, incompatible values, and conditional conflicts. For example, two plugins that both write `-kv-transfer-config` conflict even if their package names and providers differ. Two plugins sharing a read-only health provider need not conflict. The output is a deterministic, reviewable plan rather than an order-dependent sequence of mutations.

4.4 Contract Compilation and Plan Identity

The compiler normalizes version predicates and identifiers, resolves each requirement to one selected provider, topologically orders dependencies, and checks resource claims against typed ownership modes. It then expands lifecycle declarations into process-specific evidence obligations. The canonical serialization freezes plugin versions, provider choices, host compatibility, configuration digests, target roles, ordering, and failure policy under one plan ID. Every launch adds a fresh launch ID and process epoch without changing the compiled semantics.

This compilation boundary provides a limited static guarantee: an admitted plan has no unresolved requirement, dependency cycle, ghost mediator, or multiply owned exclusive resource under the declared taxonomy. It cannot show that a hook executes, that a provider mutates the intended resource, or that the taxonomy captures every semantic interaction. Those are explicit runtime or evaluation obligations rather than hidden assumptions of the compiler.

4.5 Explicit Lifecycle

Operator intent is not evidence state. Its durable vector is:

`installed, configured, enabled.`

Installed is distribution evidence; configured is validated saved input; enabled is durable operator intent. The separate evidence predicates are declared, planned, invoked, effective,

and exposed. In particular, runtime-effective means that every required plan obligation is complete, not merely that an implementation ran. Reachability, health, degradation, and compatibility are orthogonal projections. Thus an external outage does not erase intent, and saved intent does not certify a running process.

4.6 Authority- and Host-Owned Runtime Evidence

Each activation plan emits obligations keyed by launch identity, authority, process role or resource, plugin/component digest, host/runtime version, and capability. A worker (or an observer in its authority domain) records load and invocation events; provider and external obligations name their own completion sources. The aggregator verifies identity, freshness, authority, expected membership, and every declared completion predicate. Parent-process import is insufficient for a worker-side hook, and invocation alone is insufficient for a declared resource or provider effect. Evidence is compact lifecycle telemetry, not prompt content.

In the running example, the static probe rejects the wrong namespace before launch. If static discovery succeeds but one worker never invokes the hook, the plan may remain enabled and partially loaded, but cannot become globally runtime-effective. This same distinction covers inert imports and mixed worker versions without special-case status strings.

4.7 Fail-Closed Activation and Deterministic Rollback

Planning is side-effect free. Activation proceeds through prepare, validate, activate, observe, expose, and commit; shutdown and rollback are explicit terminal or compensating paths (Figure 2). Exposure is distinct from activation: a candidate may run behind a closed gate while obligations accumulate, but it cannot receive external traffic until the gate accepts its complete identity and evidence set. Missing required services, contradictory capabilities, or incomplete evidence stop before exposure. The manager stores the canonical predecessor plan, rendered inputs, version identities, and authority owners. Rollback re-renders that predecessor and requires its own evidence gate. External-service lifecycle remains operator-owned; rollback can restore client configuration without pretending to downgrade a service it does not control.

4.8 Authority Separation

Core owns manifests, intent, planning, and evidence validation. Providers own translation and provider-specific checks. The runtime owns process execution and invocation facts. External operators own services, clusters, drivers, and data.

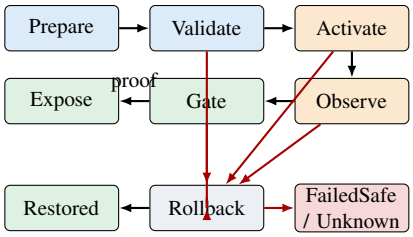


Figure 2: Activation is a transaction, not a callback list. External traffic remains closed through observation and proof validation. Any pre-exposure failure enters explicit compensation; unverifiable compensation is reported as safety-unknown rather than success.

A provider protocol exposing plan/render/check therefore cannot silently apply, delete, or restart infrastructure. This separation limits both failure blast radius and the claims that status may make.

5 Durable Coordinator

The reference coordinator is written in Python, but ECPA is not synonymous with its package or prototype host. It comprises the versioned contract, planner/evidence semantics, transactional runtime, adapter interface, corpus, and conformance tests. Core modules parse manifests, discover installed bundles, maintain user configuration, resolve compatibility, reject duplicate registrations and resource conflicts, and dispatch to host providers. The CLI exposes list, inspect, configure, enable, check, and run-oriented paths. Providers currently cover scheduler policies, KV connectors and services, memory/data-path controls, lifecycle observers, and rendered production-stack inputs. The current evaluation uses one vLLM-compatible prototype host, but all host-specific command lines, namespaces, process-role mappings, and evidence decoders reside behind the adapter interface. This separation is a design property; it is not evidence of cross-runtime portability.

Release 0.3.0 keeps static bundle discovery separate from implementation entry points and distinguishes persistent intent from provider health. Its stable Manifest 0.3 adds explicit extension dependencies and resource claims so incomplete or conflicting activation graphs fail before host launch. It rejects mutating provider plans and documents that only a process-owned observer can establish runtime effectiveness. Post-release main also composes distinct, hash-bound StateAxis MOD plans under separate configuration keys and rejects overlapping claims; this is contract and test evidence, not a performance or runtime-effect result.

Major evaluation pieces remain incomplete. The implementation has a pure contract compiler that resolves normalized public-PEP-440 requirements to exactly one provider, derives deterministic dependency order, rejects cycles, binds

typed observation authority and quorum policy, and checks exclusive, shared-read, and selected-mediator resource ownership. A frozen machine-readable taxonomy and an early modeled seed measure the static admission mechanics, with the limitations reported in Section 7. The matched nine-start partial-worker campaign now establishes that the formal protocol can collect and replay process-owned evidence, and it falsifies H1 for the tested implementation. Consequently we claim one negative H1 result, one bounded lifecycle qualification, and a 4-case partial H2 result, not a completed end-to-end ECPA evaluation.

An exact-commit, cross-repository adapter preflight exercises the host's preemption-policy controller and translates its deployment journal into a Plan-bound invocation statement. It establishes wire compatibility, not serving, worker coverage, or a formal-real result.

The coordinator persists an immutable canonical plan before launching work. The plan binds plugin and host identities, resource claims, predecessor state, and role/ordinal evidence obligations. Coordinator state separately binds the manager epoch, launch ID, observed process inventory, and evidence expiry. Generation compare-and-swap prevents a delayed actor from committing over a newer plan. Append-only intent and receipt records make recovery deterministic: after a crash, the coordinator resumes observation, retries an idempotent action, or enters fail-safe rollback; it does not infer success from the last attempted step.

The *ExposureGate* is the only path from CommitReady to Effective. It checks target-set completeness, evidence identity and freshness, unresolved conflicts, and predecessor durability. Evidence is scoped to the authority that observed it: a worker may attest hook invocation, a provider may attest a translation or check, and a deployment controller may report rollout status. No authority can substitute its acknowledgement for another authority's effect. Epoch changes invalidate old membership evidence, preventing a departed worker or replayed receipt from satisfying a new launch.

The current single-manager SQLite implementation exercises WAL recovery, receipts, replay fencing, generation CAS, predecessor gating, and explicit FailedSafe or SafetyUnknown outcomes. FailedSafe is claimed only after an actual route query proves traffic closed; an unobservable or failed closure is SafetyUnknown and blocks admission. It is a reference for transaction semantics, not a distributed consensus service. A production coordinator must additionally measure database contention, multi-host partitions, and high-availability failover.

An optional JCS/JWS Ed25519 attestation profile passes shared Python and Go vectors, but uses test keys. Signature checks do not replace epoch, coverage, or commit checks; a production issuer and trust root remain absent.

The architecture in Figure 1 has unequal observation depth across adapters. Only the vLLM host path currently feeds plan-bound process observations. Mooncake health and production-stack status can constrain provider checks, but they are not process-owned runtime-effect attestations. The external-lease interface has only fake bindings in the current reference artifact.

6 Extension Corpus and Case Studies

Our external-validity population is the 24 MODs rendered by the vLLM-HUST plugin page, frozen by website commit, metadata blob, and SHA-256 digest. We resolved every page entry to a repository numeric identity, default branch, and exact head, then inspected packaging metadata, manifests, registration functions, implementation paths, and repository-local tests. Research projects from our own portfolio are not substituted for or appended to this population.

The release audit is a separate operational inventory rather than a silent replacement for that frozen page population. At release 0.3.0 it recorded 36 distributions, 39 valid registrations, 24 activation-intent bundles, and 15 inspect-only bundles. Those counts describe Manager intake and lifecycle eligibility; they do not establish that every entry ran on an NPU, composed with every other entry, or improved performance.

Of these MODs, 16 installable distributions register the ECPA static-bundle namespace and are discoverable without importing plugin implementations. DLA exposes a concrete false-effective candidate: it ships a static bundle under the near-miss namespace `vllm.extension_bundles`, so the ECPA manager discovers no DLA bundle. Other MODs use general entry points, direct host integration, or source-patch/scaffold delivery and therefore exercise adaptation rather than native bundle discovery. Isolated source/package/CPU checks passed for 17 MODs; another 5 reached Ascend host dependencies unavailable in the audit environment. These checks establish neither NPU execution nor runtime effectiveness.

Separately, a historically reviewed 11-extension seed and 5 candidates retain a 31-case static planner oracle. That seed includes local provider examples and is used only to keep the modeled planner experiment reproducible; it is not cited as page-corpus coverage.

The unit of analysis is an extension contract, not a favored optimization. The page population crosses scheduler decisions, speculative decoding, MoE offload, KV transfer/materialization/compression, graph and communication paths, scaling, and lifecycle observability. BidKV is one scheduler-policy instance. It does not define ECPA's contract schema, evidence vocabulary, or evaluation oracle.

The source-bound corpus artifact retains all 24 entries, their exact repository heads, delivery modes, and individual package and source/CPU test outcomes. The selected cases in Table 3 illustrate the distinct effect obligations; the full inventory is not used as evidence of NPU execution or runtime effectiveness.

Table 3: Representative cases selected from the 24-MOD page population.

Case	Extension/authority	ECPA obligation
BidKV	scheduler preemption policy	scheduler-owned invocation and decision effect
DiffSpec / vSpec	speculative decoding	bundle discovery, worker coverage, accepted-token effect
LatchMoE	MoE offload	worker invocation, placement effect, and rollback boundary
KVCompress	KV compression	cache-path invocation, representation and ownership evidence
Lifecycle Profiler	general-plugin observer	required roles emit launch-bound lifecycle events
BetterScale	direct worker-class integration	exact host/pins and worker-path effect evidence
TraceLoom	scheduler/native analyzer	capture provenance separated from offline analysis
DLA	bundle plus victim selector	reject the near-miss bundle namespace before activation

The cases exercise distinct failure semantics. A speculative-decoding MOD can load while never producing an accepted draft token. An offload, compression, or scaling implementation can execute setup code without changing placement, representation, or the selected worker path. A lifecycle observer can run in the parent while one worker emits no events. Trace capture can succeed while offline analysis is incorrectly treated as runtime proof. A scheduler policy, including BidKV or DLA, can be selected without ever affecting a scheduling decision; DLA additionally demonstrates that a near-miss registration namespace can hide an otherwise packaged bundle. For each MOD, the corpus records source identity, integration class, package result, CPU/source result, and evidence limit. None is promoted to ECPA L3 merely because repository-local tests pass.

These cases become fixtures rather than anecdotes: each needs a positive cell, an inert-import or missing-process cell, an identity/version mismatch, and a conflict or authority-boundary cell. The corpus therefore supplies generality pressure to the contract while the conformance suite prevents the reference implementation from defining correctness by itself.

The page population supplies breadth across 24 MODs, multiple delivery models, plugin classes, and distinct process/authority scopes. Historical corpus metadata may mention another supported runtime, but that is not a cross-runtime ECPA experiment. Our generality evidence is therefore the combination of corpus diversity, prototype-host process scopes, the fault model, and the explicit contract/host boundary—not cross-framework validation.

The earlier 31-case modeled study retains its canonical resource-and-alias taxonomy, synthetic boundary controls, and frozen oracle as a reproducibility fixture. It is not real-MOD H2 evidence. Its detailed metrics remain in the artifact; Section 7.5 reports the separate source-bound H2 subset.

7 Evaluation Methodology and Current Evidence

The executable false-effective harness preregisters 3 matched arms: vanilla vLLM entry points, manual integration, and ECPA. Formal cells require 3 independent service starts per arm under a 3-by-3 Latin-square order. A separate oracle derives outcomes from preregistered truth predicates and raw runtime observations; implementation decisions are not accepted as ground truth. The checked r5 campaign has 9 complete formal-real starts in one scenario cell. H2 has a partial source-bound result outside that matched runtime aggregate; H3 and H4 have no completed formal results. A 45-start helper-process run tests the harness lifecycle and failure classification, but is explicitly reference-synthetic and excluded from paper aggregates. Formal ingestion is runner-owned and content-addresses the command, semantic environment, raw observations, preregistered scenario/protocol, and independently recomputed oracle. Aggregation is stratified by scenario and arm with the matched repetition as the paired unit; failure and missingness remain explicit. A fail-safe closure is reported as a rollback class, not counted as restored rollback success. The checked claim graph maps 4 falsifiable claims to their counterexamples, authority-owned facts, and non-substitutable experiments. The first formal comparison is the partial-worker-coverage scenario across all 3 arms and 9 independent starts. Its producer, deployment identity, commands, adapter registry, and 5 independent lifecycle sources were frozen before the authoritative r5 campaign. We separate four artifacts that would otherwise collapse into one "ready" bit. The study preregistration fixes the comparison and falsifier; a repository-owned producer receipt records merge and review facts; a deployment registration packet binds the matched identities and command fingerprints; and the runtime adapter registry is the only artifact that authorizes those commands. The checked deployment packet currently retains 0 blockers and has 3 of 3 arms registered. Null slots are intentional: neither a developer-machine command nor a candidate PR is inherited as deployment identity. Admission requires all arm and lifecycle source registrations in one transition, so a partially frozen comparison cannot enter the aggregate. The hypotheses are aligned with issues M0–

M3. **H1:** in preregistered failure cells, ECPA has zero false-effective outcomes and at least 99% required-event coverage. **H2:** conflict precision and recall each reach 0.95 against an independent oracle, without rejecting all compositions. **H3:** every unsupported tested tuple fails before serving and at least 95% of injected upgrade/rollback failures meet the declared recovery objective or bounded operator action. **H4:** relative to manual integration, added median startup cost is at most 5% or 250 ms, throughput regression at most 1%, and p99 request-latency regression at most 2% in every supported formal cell.

7.1 Release Qualification Outside the Formal Aggregate

The release-readiness artifact records 9 passed gates for release 0.3.0. One gate is backed by a clean-wheel run of the Manager and KV-materialization plugin on a pinned current vLLM-HUST host, vLLM-Ascend compatibility patch, Qwen3-0.6B, and one Ascend 910B2 device. The host served two HTTP requests, and the live engine process emitted two plan/launch-bound `runtime_effective` receipts tied to those request identities. After SIGINT to the Manager supervisor, the API and engine exited, port 8100 became bindable, device memory returned to the recorded idle baseline, and status stopped projecting runtime effectiveness. Disable, forget, and uninstall then removed intent and discovery without stopping an external service.

This is 1 lifecycle qualification, not the now-complete formal H1 cell. It has no matched vanilla or manual arm, repeated-start schedule, injected partial-worker fault, latency distribution, or throughput measurement. We therefore use it to retire the claim that no real host path has executed; it does not change the H1 result or measure rollback fault rates and performance overhead.

7.2 Baselines and Workloads

We compare (1) vanilla vLLM entry-point discovery/loading [22, 24], (2) manual integration using documented flags, configuration, and operator checks, and (3) ECPA. Arms pin identical host, plugin, provider, model, workload, hardware, process topology, and warm/cold state. Formal cells use at least 3 independent service starts and alternating arm order. Agent, repeated-prefix, and online chat workloads cover different lifecycle and connector behavior; CPU-only fixtures isolate planner correctness.

7.3 Matrices and Metrics

The failure matrix injects missing distributions, malformed manifests, unsupported versions, provider ambiguity, parent/worker mismatch, inert hooks, service outage, replayed evidence, partial rollout, process crash, and interrupted rollback. The composability matrix covers single plugins, compatible and incompatible pairs, and three-plugin sets across flags, hooks, schedulers, connectors, services, and provider translations. The version matrix crosses current, previous, and next host versions with below/in/above plugin ranges, old manifests, and mixed workers.

Table 4: Authoritative r5 partial-worker-coverage result. "PASS" means the record passed evidence validation; it is not the H1 hypothesis verdict.

Arm	Starts	False-effective	Coverage
Entry points	3	3/3	0.5
Manual	3	3/3	0.5
ECPA	3	3/3	0.5

Primary metrics are activation-event coverage, false-effective rate, conflict precision/recall, rollback success and recovery time, unsupported-tuple admission, and cross-process agreement. Costs include phase-separated startup time, throughput, latency quantiles, CPU time, memory, process messages, and evidence bytes. Raw observations follow the checked-in JSON schema; failures remain rows rather than being removed from aggregates.

Each experiment also targets a premise of the safety argument. Worker replacement and partial coverage challenge frozen epoch membership. Replayed receipts, cross-source substitution, and SUT self-report challenge identity and authority locality. Crashes before and after route mutation challenge the intent–effect–query–receipt refinement. Lease expiry and controller recreation challenge stale-writer exclusion. Query loss and compensation failure challenge conservative recovery. A cell is informative only if the injection is independently observed; asking the SUT whether its own fault occurred cannot falsify the corresponding premise.

The static planner and transaction model have only modeled results; the source-bound page corpus, partial H2 subset, and one NPU lifecycle qualification are observed but outside the matched runtime aggregate. The first matched runtime cell is complete. H3 recovery and H4 overhead have no completed formal results, with 1 complete H1 scenario cell and 3 registered adapters.

All 9 starts are validator-approved and contain the five registered phases: service readiness, workload completion, fault injection, observer capture, and shutdown. The post-cleanup batch replay also passed. The fault actuator removed one of two worker evidence journals, producing 0.5 coverage. Every arm nevertheless retained an effective claim, so H1's ECPA falsifier fired in all three ECPA repetitions. Equal outcomes across arms mean this implementation did not prevent false-effective state in the tested partial-coverage cell. The fault removes evidence, not worker execution; the result therefore concerns evidence coverage and the recorded effec-

tiveness claim, not traffic exposure or whether computation stopped. The r2–r4 campaigns are excluded from this aggregate. Recorded startup durations are lifecycle fields for H1 and are not interpreted as H4 performance measurements.

7.4 H2 Redesign: Real MOD Composition

H2 becomes the main composition evaluation; H1, H3, and H4 respectively test effect truthfulness, behavior under failures, and system cost. The expanded protocol is prospective. It preserves the original H2 precision and recall target of 0.95 and does not relabel historical source or runtime data.

The unit is a pinned set of MOD implementations, host/backend revisions, configuration, topology, workload, and installation order. We first audit the full frozen population, then select cases across interface, version/symbol, configuration, shared-hook, shared-state, and external-authority surfaces. Every case separates original delivery from ECPA adaptation. A discovery set informs the contract vocabulary; a held-out set grouped by MOD family is frozen before final predictions to prevent a reordered variant of a known pair from masquerading as generalization.

An independent oracle records compatible, incompatible, or unresolved with supporting source and execution predicates. Its labels and diagnostic verdicts are unavailable to the prediction path. Source-only labels concern stated constraints, not native semantics. We compare package/version checks, documented manual integration, and full ECPA, with additional ablations for resource normalization and runtime effect checks. Manual integration receives the same source access and a declared effort budget, not a deliberately broken configuration.

Positive runtime cells require each singleton to work and the composition to preserve both intended effects. Both registration orders are tested; public request success alone is insufficient. A compression composition must cross its real activation threshold, and an observer must cover the relevant processes. Missing hardware or a missing provider remains unsupported or unresolved. Early rejection is evaluated separately from safe serving, and source-level precision cannot be presented as runtime correctness.

We report definite-case precision/recall together with abstention coverage, compatible-case admission, and unsafe admissions, including all unknowns in the inventory. Adaptation is measured as changed files/lines, host changes, human effort, and reuse on a held-out revision. Native comparisons retain three matched arms, three independent starts, and the original hardware and workload controls. This prospective design alone adds no H2 result.

A separate source-fixed KVCompress–Profiler development block observed ten HTTP completions across five groups and fifteen original drained recording shards (including empty shards). This is one family, not a three-method, held-out, legacy-score, performance or quality result. The archived r6 packet retains source/adapter costs, bounded physical observations and prior unknowns; no historical record is relabeled.

7.5 Partial Source-Bound H2 Result

The successor H2 corpus currently has 4 definite, independently labeled compositions for which the Manager also produced a binary decision. Recomputing predictions against the frozen oracle gives $TP = 2$, $TN = 2$, $FP = 0$, and $FN = 0$. Admit precision/recall and reject precision/recall are therefore all 1.0. The admit prevents a reject-all policy from satisfying H2; that baseline has admit recall 0.

The first true positive is Lifecycle Profiler plus KVCompress on an exact TP2 vLLM-HUST/Ascend stack. Both frozen installation orders completed a short context request and produced process-owned host evidence with clean shutdown. The scored workload has `max_model_len=2048`, below KVCompress's 9216-token threshold. This establishes compatible host execution, not an observed compression transaction. Two true negatives are the exact-stack KNorm plus KVCompress pre-launch rejection and a source-bound three-MOD rejection containing import-only KNorm. Production Stack plus standalone KEDA is unscored: its provider consumes a supplied ownership-conflict report. With that report removed, the same configuration returns unknown. This source-bound check establishes report consumption, not independent detection.

The second true positive is a source-bound, pre-launch admit for Pipeline Microbatch plus BidKV on exact vLLM-HUST commit `3e57b2f75cffffd26dda0df4d9bcac890cfe99fc`. Its independent oracle was frozen before the Manager prediction. The frozen Host source exposes separate batch-admission-policy and preemption-policy controllers, protocol versions, and scheduler fields, and the Manager projects the pair onto those distinct options. This is source-only evidence. The qualified Pipeline Microbatch cell requires PP2+TP2 on four devices, while the current allocation has two; we did not launch the Host and make no runtime or performance claim for this case.

A separate real TP2 qualification crosses the compression threshold in both of its 2 installation orders. Its independently recomputed raw evidence binds a 1201-token semantic request to a 1024-token physical KV budget, one EngineCore epoch, 2 distinct acknowledgements from TP0 and TP1, an HTTP response, and empty owned-process, port, and NPU state after shutdown. We classify this as `QUALIFIED_NONFORMAL`: it is excluded from the H2 score and provides no performance, compression-quality, or correctness comparison. The campaign field labeled `frozen_utc` is later than the observed first launch, so we explicitly reject it as a preregistration timestamp; pre-launch integrity is limited to the runner's digest checks of the cam-

paign, oracle, and runner.

This is a partial H2 result. 4 scored cases cannot estimate prevalence or support the preregistered population claim. The CLI/config collision remains unresolved because no second qualifying active bundle claims the same launch projection. Two communication/process-carrier cases retain independent *abstain* labels and remain unscored. The external-authority check is report-only and pre-launch, without a Kubernetes runtime receipt. The conditional DiffSpec/KVCompress pair is also excluded pending physical-ownership and installation-order review. We therefore report the observed 1.0 values with $n = 4$ and keep H2 incomplete.

7.6 Supporting Gate and Recovery Evidence

The SQLite reference gate exercises synthetic attribution, crash recovery, draining, and rollback. A separate real Qwen2.5-7B-Instruct TP2 qualification uses an independent HTTP authority: quarantining one worker's journal blocks candidate traffic, restoring it admits a completion, and a second quarantine revokes admission. This establishes an invocation-coverage boundary, not general semantic correctness or the H1 prevention claim.

Separate native transport, KV-byte, and source-drained attention trials retain their full raw journals, failures, and exact source identities in the artifact. They qualify only the declared finite observations; numerical correctness, quality, and performance remain unestablished. The earlier controller-crash window retained 60 single-attempt requests with deadline loss 0/60, but kept the predecessor model alive. It is control-route recovery, not model-backend restart. Baselines, complete objectives, and native replacement remain missing; these trials are `QUALIFIED_NONFORMAL` and do not populate the formal H3/H4 results.

7.7 Remaining Evidence Gaps

The current contribution is a source-grounded composition problem, implemented contract/planning machinery, and an incomplete evaluation. H1 remains falsified for the tested implementation; H2 retains only the 4 definite cases above. The expanded H2 study, traffic prevention, recovery, and overhead remain unestablished. Source snapshots, modeled checks, and observed finite qualifications retain distinct scopes.

8 Related Work

Inference and module plugins. vLLM's Plugin System loads registered general plugins during process initialization, while Endpoint Plugins extend the API server [23,24]. These are the nearest loading mechanisms, and ECPA uses rather than replaces them. Their registration and load events do not define a cross-process target set, exclusive resource plan, or transaction commit criterion. OSGi offers mature bundle lifecycles and a dynamic service registry [16]; ECPA differs in binding a selected plan to fresh evidence from a finite inference-process epoch.

Inference-system context. PagedAttention and PRETZEL optimize serving-engine memory and prediction pipelines [12, 13]. DistServe separates prefill from decoding, while Llumnix migrates requests and their in-memory state across instances [19, 27]; Sarathi-Serve changes the scheduling of prefill and decode work, and dLoRA dynamically places and merges LoRA adapters [1,26]. These results motivate the heterogeneous scheduler, KV, process, and routing surfaces that independently packaged extensions now modify. Our question is narrower than improving those algorithms: whether a declared extension plan has actually taken effect in every required process before its generation receives traffic. Accordingly, the planned overhead comparison must hold the host algorithm and workload fixed across loading, manual integration, and ECPA arms; it cannot claim the throughput improvements reported by these serving systems.

Control-plane reconciliation. Acto tests whether operators reach declared end states, while Anvil verifies eventual stable reconciliation [8, 20]. Envoy xDS ACK/NACK concerns configuration acceptance [6]; specialized Kubernetes lifecycles and observed-state conventions likewise require application-specific effect predicates [5, 10, 11].

Distributed predicates. D3S instruments process-local states and checks predicates over global snapshots, with ordering, membership and failure handling [14]. ECPA does not introduce cross-process predicate checking. Its proposed distinction is compiling extension declarations into fixed, authority-specific obligations bound to generation admission. A competent predicate monitor with the same observations and routing gate could enforce the same predicate; this stronger comparator remains unmeasured. H1 falsifies the tested implementation's prevention claim.

Snapshots, updates and upgrades. Chandy–Lamport snapshots establish global-state observation for stable-property detection [2]. ECPA's finite source-drained ACK is neither the full snapshot algorithm nor proof of continuing semantic effectiveness. Consistent network updates coordinate versioned packet exposure [18]; ECPA's intended unit is an extension plan, without per-packet consistency. Kitsune explicitly transforms live program state [9]; ECPA neither rewrites live code nor proves state-transformer correctness.

Runtime hooks and provenance. OCI runtime hooks provide ordered extension points around container lifecycle [15]; they do not negotiate semantic capabilities or prove application-level invocation. in-toto binds software-supply-chain steps and artifacts to authorized actors [21]; CamFlow captures whole-system data provenance at the OS boundary [17]; and SPIFFE/SPIRE issue workload identities [3, 4]. These systems can authenticate ECPA artifacts and evidence

producers. They do not by themselves determine whether a planned extension became effective in a frozen set of serving processes. Leases provide the historical time-bounded authority primitive [7]; ECPA uses epochs and fences for freshness but does not claim a new lease protocol. ECPA does not claim these systems' capture, cryptographic, identity, or consistency properties.

9 Limitations and Discussion

ECPA 0.1 is a candidate interoperability specification, not an industry standard. Its cumulative levels distinguish static discovery (L0), schema and namespace validity (L1), conflict-free negotiated plans (L2), process-owned runtime evidence (L3), and injected rollback verification (L4). Conformance is always for a pinned runtime/plugin/process cell; passing L1 cannot be advertised as runtime compatibility.

The contract is versioned independently of the Extension Manager. Reverse-DNS capability and resource namespaces identify ownership. Optional additive fields may evolve within a compatible revision, whereas removal, semantic change, or tighter accepted values require a major change and a deprecation window. Providers remain translators and observers; adopting the manifest does not transfer runtime or external-service authority.

Standardization requires independent maintained host/extension adoption, public RFC feedback, interoperability between independent implementations, compatibility history, external users and neutral governance. Until those evidence gates pass, "standardizable contract" is the strongest claim.

9.1 Evidence and External Validity

Evidence has cost and can itself fail. A synchronous per-request proof would be unacceptable; the design therefore targets lifecycle and sampled invocation events, but H4 must show that this does not move meaningful work onto the request path. Process membership is also subtle under elastic workers. The plan must declare whether membership is fixed, epoch-based, or quorum-based; otherwise a disappearing worker could make success vacuous.

Conflict taxonomies can be incomplete. Precision and recall are measured against an independently labeled oracle, but that oracle may miss semantic interactions. The current real-source subset has only 4 scored cases and does not cover several required conflict surfaces. We mitigate this with explicit unknown/unverified outcomes and negative cells, not a universal composability claim. Deterministic rollback can restore manager-owned inputs, not external data or services; bounded operator action is therefore a first-class outcome.

The single-host SQLite coordinator implements durable plans, receipts, epochs, fenced replay, generation CAS and rollback. Deterministic traces test these semantics; one pinned vLLM-HUST/Ascend launch demonstrates native observers and owned cleanup. Narrow TP2 qualifications cover invocation gating, KV metadata/bytes and native argument completion. None establishes production attestation, elastic membership, formal effectiveness, recovery or overhead. SQLite remains an implementation choice. Stop or reframe the paper as an engineering report if runtime intent cannot be separated from effect, conflict thresholds remain unmet after one correction cycle, rollback loses the predecessor, or overhead exceeds the frozen bounds.

The executable model separates intent, planning, runtime evidence, provider and external-service authorities. Append-only WAL intents/receipts, epoch-bound attestations and generation CAS support fenced logical-once replay; rollback is conditional on ownership and may fail safe. This adversarial design oracle establishes no production control plane, distributed transaction, physical exactly-once guarantee or performance result.

External validity is limited to this runtime. Multi-extension, process-role, version, topology and fault tests can show breadth here; cross-runtime adoption remains future work. Accordingly, neither this paper nor the 24-MOD page population supports a cross-framework or industry-standard claim.

10 Conclusion

Inference extension composition is constrained by heterogeneous interfaces, implicit host dependencies, configuration predicates, and shared runtime ownership. ECPA makes known requirements explicit through contracts and plans, then binds effect claims to the authorities that observe execution. The real-source audit also exposes its limits: a descriptor is not a host integration, an undeclared wrapper is not automatically detected, and static acceptance does not prove semantic compatibility. The expanded H2 study therefore evaluates original MODs, adaptation effort, independent conflict labels, and preserved runtime effects together. Existing evidence remains limited: the first H1 implementation was falsified, H2 has only 4 definite cases, and recovery and overhead validation are incomplete. The composition-centered study is the path to testing the system's intended contribution, not an already established result.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, pages 117–134, 2024.

[2] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[3] Cloud Native Computing Foundation. Spiffe standards. https://spiffe.io/docs/latest/spiffe-about/overview/. Official documentation, accessed 2026-09-18.

[4] Cloud Native Computing Foundation. Spire concepts. https://spiffe.io/docs/latest/spire-about/spire-concepts/. Official documentation, accessed 2026-09-18.

[5] Container Storage Interface Authors. Container storage interface specification. https://github.com/container-storage-interface/spec/blob/master/spec.md. Official specification, accessed 2026-09-18.

[6] Envoy Project. xds rest and grpc protocol. https://www.envoyproxy.io/docs/envoy/latest/api-docs/xds_protocol. Official protocol documentation, accessed 2026-09-18.

[7] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*, pages 202–210, 1989.

[8] Jiawei Tyler Gu, Xudong Sun, Wentao Zhang, Yuxuan Jiang, Chen Wang, Mandana Vaziri, Owolabi Legunsen, and Tianyin Xu. Acto: Automatic end-to-end testing for operation correctness of cloud system management. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, pages 96–112, 2023.

[9] Christopher M. Hayden, Edward K. Smith, Michail Denchev, Michael Hicks, and Jeffrey S. Foster. Kitsune: Efficient, general-purpose dynamic software updating for c. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, pages 249–264, 2012.

[10] Kubernetes Authors. Device plugins. https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/. Official documentation, accessed 2026-09-18.

[11] Kubernetes Authors. Kubernetes api conventions. https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md. Official conventions, accessed 2026-09-18.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023.

[13] Yunseong Lee, Alberto Scolari, Byung-Gon Chun, Marco Domenico Santambrogio, Markus Weimer, and Matteo Interlandi. Pretzel: Opening the black box of machine learning prediction serving systems. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, pages 611–626, 2018.

[14] Xuezheng Liu, Zhenyu Guo, Xi Wang, Feibo Chen, Xiaochen Lian, Jian Tang, Ming Wu, M. Frans Kaashoek, and Zheng Zhang. D3S: Debugging deployed distributed systems. In *5th USENIX Symposium on Networked Systems Design and Implementation*, 2008.

[15] Open Container Initiative. Oci runtime specification: Lifecycle and hooks. https://github.com/opencontainers/runtime-spec/blob/main/config.md#posix-platform-hooks. Official specification, accessed 2026-09-18.

[16] OSGi Alliance. Osgi core release 8 specification. https://docs.osgi.org/specification/osgi.core/8.0.0/, 2020.

[17] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. Practical whole-system provenance capture. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*, pages 405–418, 2017.

[18] Mark Reitblatt, Nate Foster, Jennifer Rexford, Cole Schlesinger, and David Walker. Abstractions for network update. In *Proceedings of the ACM SIGCOMM 2012 Conference*, pages 323–334, 2012.

[19] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, pages 173–191, 2024.

[20] Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. Anvil: Verifying liveness of cluster management controllers. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, pages 649–666, 2024.

[21] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019.

[22] vLLM Project. Architecture overview. https://docs.vllm.ai/en/latest/design/arch_overview/. Accessed 2026-09-17.

[23] vLLM Project. Endpoint plugins. https://docs.vllm.ai/en/latest/design/plugin_system/. Official documentation, accessed 2026-09-18.

[24] vLLM Project. Plugin system. https://docs.vllm.ai/en/latest/design/plugin_system/. Accessed 2026-09-17.

[25] vLLM Project. Security. https://docs.vllm.ai/en/latest/usage/security/. Accessed 2026-09-17.

[26] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, pages 911–927, 2024.

[27] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, pages 193–210, 2024.